

面向 Agent 工作流的企业级知识中台：整体方案（落地版）

0. 文档说明

本文档是在“通用型产品知识库平台”前期讨论基础上的整体方案升级版，目标不是讲概念，而是形成一份**可落地、可靠、可实现、可分期建设**的知识中台方案。

文档重点解决四件事：

1. 这个平台到底要解决什么问题
2. 平台应该做成什么样，边界在哪里
3. 怎么分阶段建设，避免大而空
4. 如何确保它能真正服务 Agent，而不是再造一个文档墓地

1. 一句话定义

这是一个**面向 Agent 工作流的企业级知识中台**，用于统一承载产品、业务、规则、流程、案例、FAQ、接口说明等知识资产，并通过治理、检索、引用、权限与运营机制，为 Agent、业务系统和人工协作提供可信知识服务。

更直白一点：

- 它不是网盘
- 它不是普通 CMS
- 它不是只会上传文档的“知识库”
- 它是企业内部“知识生产—治理—服务—优化”的中枢系统

2. 为什么必须做成“知识中台”，而不是简单知识库

如果目标是服务 Agent workflow，那平台不能只回答“有没有文档”，而必须回答下面这些问题：

- 哪些知识是官方有效版本？
- 哪些知识适合给哪个 Agent 用？
- 哪些知识只能给哪些角色看？
- 检索不到或答错时，怎么定位问题？
- 知识过期后，谁来维护？
- Agent 的回答能否展示证据链和出处？

普通文档库解决的是“存储问题”；知识中台解决的是：

- **可沉淀**：知识进入平台
- **可治理**：知识变成可信资产
- **可服务**：知识可被 Agent 和系统可靠调用
- **可运营**：知识效果可以持续优化

3. 平台总体建设目标

建议目标拆成 4 个层次。

3.1 业务目标

- 建立统一知识底座，减少各部门知识分散、口径不一致的问题
- 提升产品、客服、运营、售前、交付等场景的信息获取效率
- 降低新人培训成本与组织知识流失风险
- 为企业级 Agent、Copilot、智能问答、流程自动化提供统一知识能力

3.2 AI 目标

- 为 Agent 提供结构化、可信、可检索、可引用的知识供给
- 支持不同 Agent 按业务域、权限和策略调用不同知识
- 提升 AI 回答准确率、可解释性与可控性
- 建立可观测的检索与问答优化闭环

3.3 管理目标

- 建立知识责任制、审核制、版本制、生命周期管理机制
- 建立统一的权限体系、审计体系、安全与合规能力
- 建立知识质量考核和运营看板

3.4 技术目标

- 建设统一知识模型
- 支持结构化知识和非结构化知识并存
- 支持全文检索、语义检索与混合检索
- 支持 API / SDK / Workflow 节点方式对外提供能力

- 支持多租户或多业务域扩展

4. 建设范围与边界

4.1 本平台要覆盖的内容

A. 知识接入

- 手工录入
- 批量导入
- 第三方文档同步
- API 数据接入
- 周期性同步

B. 知识处理

- 解析
- 清洗
- 切片
- 标签化
- 向量化
- 去重
- 分类归档

C. 知识治理

- 审核发布

- 版本管理
- 来源追踪
- 责任人机制
- 生命周期管理
- 质量评分与维护任务

D. 知识服务

- 搜索
- 推荐
- Agent 检索服务
- 引用回传
- 场景化调用

E. 平台后台

- 空间管理
- 权限管理
- 数据源管理
- 模型配置
- 日志审计
- 运营分析

4.2 平台不建议一期承担的内容

为了避免一开始就过重，一期不建议把这些做满：

- 复杂知识图谱自动构建
- 视频/音频多模态深度理解
- 跨区域多集群高可用方案

- 面向外部客户的开放型 SaaS 市场
- 完整低代码 Agent 编排平台

这些可以预留接口，但不建议第一阶段全上。平台不是越大越好，越能落地越好。

5. 关键假设与前置判断

在没有你业务细节输入的前提下，我先按企业通用场景做合理假设：

5.1 典型使用场景

- 内部知识问答助手
- 客服/售后知识支撑
- 售前销售答疑助手
- 运营与实施 SOP 辅助
- 产品资料统一查询
- Agent workflow 节点的 RAG 知识召回

5.2 典型知识类型

- FAQ
- 产品功能说明
- 规则制度
- 流程 SOP
- 接口文档
- 业务案例

- 术语词典
- 参数配置说明
- 培训材料

5.3 典型角色

- 平台管理员
- 知识管理员 / 知识运营
- 录入人员 / 编辑人员
- 审核人员
- 业务负责人
- Agent 配置人员 / AI 产品经理
- 普通使用者
- 审计与安全人员

这套假设足够支撑一期方案设计。如果后续你有明确行业属性，比如金融、政务、医疗、电商、游戏、制造，我可以再把方案做行业化收敛。

6. 整体设计原则

6.1 先知识建模，再功能堆叠

不要上来先画页面。先定义：有哪些知识对象、字段、关系、状态，再决定产品形态。

6.2 结构化和文档化并行

不能只做文档系统，也不能只做表单系统。必须同时支持：

- 文档型知识
- 结构化知识
- 片段级知识

6.3 治理优先于智能

先把来源、版本、责任人、审核、权限理顺，再谈多高级的智能召回。否则 AI 越强，胡说越稳定。

6.4 Agent 调用必须可解释

必须能回答：

- 调到了什么
- 为什么调到
- 用的是哪一版
- 为什么最终引用它

6.5 可扩展但不超建

一期只做闭环，把接口和能力边界设计好；不要为了想象中的未来把当前做成航母地基。

6.6 安全与权限从 Day 1 设计

权限不是“最后加个拦截器”就完事。知识系统一旦服务多个部门和 Agent，没有前置权限设计很容易翻车。

7. 平台用户角色与职责分工

7.1 平台管理员

职责：

- 平台级配置
- 租户 / 组织 / 空间管理
- 模型与检索配置
- 安全审计与系统运维协同

7.2 知识管理员 / 运营

职责：

- 建空间、建目录、建模板
- 维护分类、标签、词库
- 处理维护任务
- 监控知识质量和平台指标

7.3 录入 / 编辑人员

职责：

- 创建知识内容
- 补充结构化字段
- 提交审核
- 接收退回修改

7.4 审核人员

职责：

- 审核内容合法性、准确性、完整性
- 发布或退回
- 审核重要更新版本

7.5 业务负责人

职责：

- 对某知识域最终负责
- 决定生效范围和更新优先级
- 审批关键策略与规则类知识

7.6 Agent 配置人员 / AI 产品经理

职责：

- 配置 Agent 绑定哪个知识空间
- 配置检索策略、Prompt 策略
- 调试召回和引用效果
- 评估问题命中率和回答质量

7.7 普通使用者

职责：

- 搜索和查看知识
- 提交反馈、纠错、收藏
- 在业务系统中消费知识结果

7.8 审计 / 安全人员

职责：

- 查看操作日志
- 审核敏感数据访问
- 制定脱敏和权限策略

8. 平台总体能力架构

建议按“六层架构”建设。

8.1 接入层

负责把知识接进平台。

能力包括：

- 手工录入
- 批量导入
- 文档解析
- 外部同步连接器
- API 接入
- 定时同步任务

输入来源包括：

- Word / PDF / Excel / Markdown / HTML / CSV
- Confluence / Notion / 飞书文档 / 企业微信文档
- Git 仓库 / Wiki / 内部系统数据库 / CRM / 工单系统

8.2 处理层

负责把原始内容变成可用知识。

能力包括：

- 文本抽取
- 清洗标准化
- 自动分段与切片
- 元数据抽取
- 标签推荐
- 相似去重

- 向量生成
- 索引构建

8.3 治理层

负责知识可信与可控。

能力包括：

- 审核发布
- 版本对比
- 回滚
- 生命周期管理
- 来源与责任人管理
- 质量校验
- 冲突检测
- 维护工单

8.4 服务层

负责把知识提供给人和系统使用。

能力包括：

- 全文检索
- 语义检索
- 混合检索
- 重排
- 推荐
- 引用返回

- 对外 API / SDK
- Workflow 节点调用

8.5 运营层

负责监控与优化。

能力包括：

- 检索分析
- 问答命中率分析
- 零结果分析
- 热门知识排行
- 过期知识提醒
- 维护绩效统计
- Agent 调用效果追踪

8.6 管理层

负责后台管理与安全控制。

能力包括：

- 用户角色权限
 - 数据源管理
 - 模型配置
 - 检索策略配置
 - 操作审计
 - 安全与合规管理
-

9. 平台核心业务对象模型

这里不展开到字段级，但先给出平台必须统一的对象模型。

9.1 核心对象

1. Space (知识空间)
2. Document (知识文档)
3. Chunk (知识片段)
4. Category (分类)
5. Tag (标签)
6. Template (录入模板)
7. DataSource (数据源)
8. SyncTask (同步任务)
9. ReviewTask (审核任务)
10. Version (版本)
11. AgentBinding (Agent 绑定关系)
12. RetrievalPolicy (检索策略)
13. CitationRecord (引用记录)
14. Feedback (用户反馈)
15. MaintenanceTask (维护任务)
16. PermissionPolicy (权限策略)
17. AuditLog (审计日志)
18. QueryLog (检索日志)
19. EvaluationSet (评测集)
20. EvaluationResult (评测结果)

9.2 三层知识对象建议

必须明确三层对象，不然后续很难做准：

文档层 Document

适合：

- 原始知识载体
- 展示给人看的完整内容

片段层 Chunk

适合：

- 检索召回
- Agent 上下文拼装
- 引用定位

结构化层 Knowledge Card / Field Model

适合：

- FAQ
- 参数说明
- 规则条件
- 流程节点
- 产品功能信息

这三层一起存在，平台才既能“读”，又能“算”，还能“精确调用”。

10. 知识分类体系建议

建议至少建立四维分类。

10.1 业务域分类

例如：

- 产品
- 客服
- 售前
- 销售
- 运营
- 交付
- 技术支持
- 内部制度

10.2 知识类型分类

例如：

- FAQ
- SOP
- 规则
- 说明文档
- 案例
- 接口
- 术语

- 公告

10.3 适用对象分类

例如：

- 内部员工
- 客服坐席
- 销售顾问
- 运营人员
- 管理员
- 特定 Agent

10.4 生命周期分类

例如：

- 草稿
- 审核中
- 已发布
- 已过期
- 已下线

这个分类体系后续会直接影响：

- 后台筛选
- 检索策略
- 权限策略
- Agent 绑定策略
- 运营统计口径

11. 核心模块设计（落地版）

下面是建议的一期到二期都适用的模块结构。

11.1 模块一：知识空间管理

解决问题

把不同业务域、组织、敏感级别的知识隔离开。

核心功能

- 新建空间
- 空间层级管理
- 公共空间 / 私有空间
- 目录树配置
- 空间负责人设置
- 空间权限配置
- 空间级检索策略配置

落地建议

一期先做“业务域空间 + 权限隔离 + 负责人机制”即可，不必一开始做复杂多租户逻辑。

11.2 模块二：知识录入与采集中心

核心功能

- 富文本录入
- 模板化录入
- 文件上传导入
- 批量导入任务
- 导入预校验
- OCR 接入预留
- 第三方文档同步预留

落地建议

一期建议支持 3 类录入：

1. 手工文档录入
2. FAQ 表单录入
3. 文件批量导入

别一开始就把所有外部系统同步全做完，连接器是个无底洞。

11.3 模块三：知识编辑与结构化建模

核心功能

- 富文本编辑器
- 字段表单编辑器
- 模板选择

- 标签与分类
- 关联知识设置
- 附件管理
- 片段预览与重切片

落地建议

建议优先定义 5 个模板：

- FAQ 模板
- 产品功能说明模板
- SOP 模板
- 规则模板
- 案例模板

这样后续原型、字段和审核流程都会更稳。

11.4 模块四：审核发布与版本中心

核心功能

- 草稿保存
- 提交审核
- 审核通过 / 退回
- 发布上线
- 历史版本查看
- 版本 diff 对比
- 回滚

- 生效/失效时间设置

落地建议

一期至少做单级审核；如果你的业务强监管，再升级为多级审核。

注意：**草稿态知识绝不能直接进入 Agent 可用知识池。**

11.5 模块五：检索与召回中心

核心功能

- 全文搜索
- 多条件筛选
- 语义搜索
- 混合召回
- 相似问题推荐
- 检索结果高亮
- 结果重排
- 无结果兜底

Agent 特有功能

- 返回 chunk 级结果
- 返回版本、来源、可信度
- 返回是否可引用
- 返回可见权限范围

落地建议

一期采用“关键词 + 向量 + 基础重排”的混合检索最现实，效果和复杂度相对平衡。

11.6 模块六：Agent 绑定与知识服务中心

这是平台灵魂模块

核心功能

- Agent 列表管理
- Agent 绑定知识空间
- Agent 绑定模板/标签/知识类型
- Agent 绑定检索策略
- Prompt 策略配置
- workflow 节点知识调用配置
- 对外 RAG API
- 引用回传
- 调试与回放

落地建议

一期必须把下面 4 个能力做出来：

1. Agent 选哪些空间
2. 每次检索调什么策略
3. 返回哪些引用字段
4. 调试页能看到召回结果

如果没有调试台，后面答得不准时大家只会互相甩锅，场面会很互联网。

11.7 模块七：知识维护与质量中心

核心功能

- 过期提醒
- 未更新提醒
- 低命中识别
- 重复知识识别
- 冲突知识识别
- 缺失字段校验
- 维护任务派发
- 责任人看板

落地建议

一期先做：

- 过期提醒
- 长期未更新提醒
- 缺字段校验
- 维护任务

先把维护闭环跑起来，比一上来做 fancy 质量分更值。

11.8 模块八：权限与安全中心

核心功能

- 角色权限管理
- 空间权限
- 文档权限
- 字段权限预留
- 数据分级
- API 鉴权
- 敏感信息识别
- 审计日志
- 防导出策略预留

落地建议

一期权限模型建议：

- RBAC 为主
- 空间 + 文档级控制
- 字段级权限先预留

别直接搞复杂 ABAC 全套，一期容易把自己绊倒。

11.9 模块九：运营分析中心

核心功能

- 搜索次数

- 命中率
- 零结果率
- 热门知识
- 热门问题
- Agent 调用量
- 引用成功率
- 用户反馈统计
- 待补知识建议

落地建议

一期先聚焦 6 个指标：

- 检索次数
- 检索命中率
- 零结果率
- Agent 引用率
- 负反馈率
- 过期知识占比

11.10 模块十：系统后台与配置中心

核心功能

- 模型配置
- 向量模型配置
- 重排模型配置
- 检索参数配置

- 词库与同义词管理
- 通知配置
- API Key 管理
- Webhook 配置
- 系统日志与审计

落地建议

一期后台配置尽量“默认可用 + 少量高级项”，不要把 200 个参数直接丢给运营。

12. 面向 Agent 的关键设计方案

这一章是整个平台最关键的部分。

12.1 Agent 与知识库的关系模型

建议建立以下映射：

- Agent → 知识空间
- Agent → 知识类型
- Agent → 检索策略
- Agent → Prompt 模板
- Workflow 节点 → 知识调用服务
- 用户角色 → 最终知识可见范围

12.2 标准检索调用链路

建议调用链路如下：

1. 用户提问进入 Agent
2. Agent 判断是否需要知识检索
3. 将 query 和上下文传给知识服务
4. 知识服务根据 Agent 绑定策略限定检索范围
5. 执行 query rewrite（可选）
6. 执行关键词召回 + 向量召回
7. 合并结果并重排
8. 权限过滤
9. 返回候选 chunk + 引用元数据
10. Agent 选择引用内容拼装回答
11. 输出答案 + 引用来源 + 版本号
12. 用户反馈结果写回日志与优化系统

12.3 Agent 侧必须拿到的引用信息

建议返回字段包括：

- 文档标题
- chunk 内容
- 文档 ID
- chunk ID
- 知识空间
- 版本号
- 发布时间

- 来源类型
- 来源链接 / 原文入口
- 责任人
- 可信等级
- 是否允许展示给最终用户

12.4 Agent 调试台必须具备的能力

- 输入 query 测试
- 查看 query rewrite 结果
- 查看召回结果列表
- 查看重排分数
- 查看最终被引用片段
- 查看过滤原因（权限/状态/过期）
- 查看回答效果与反馈

这是后续你做“知识库与 Agent 链接展示”的核心页面之一，必须做。

13. 核心流程设计

13.1 录入到发布流程

1. 选择空间
2. 选择模板或上传文件
3. 录入正文与结构字段

4. 设置标签、分类、责任人、有效期
5. 系统预校验
6. 提交审核
7. 审核通过后发布
8. 进入检索索引与 Agent 可用池

13.2 知识维护流程

1. 系统发现过期 / 长期未更新 / 负反馈高
2. 自动生成维护任务
3. 分配给责任人
4. 修改后重新提交审核
5. 发布新版本
6. 旧版本归档

13.3 Agent 调用流程

1. Agent 接到任务
2. 判断调用知识服务
3. 根据 Agent 配置限定知识空间和策略
4. 返回候选知识
5. 生成带引用结果
6. 写入 query log、citation log、feedback log

13.4 用户反馈闭环流程

1. 用户查看答案或知识详情
2. 点赞 / 点踩 / 标记无效 / 提交纠错
3. 系统记录反馈
4. 低质量知识进入维护池
5. 运营人员复盘优化

14. 非功能需求：可靠、可实现，靠这部分兜底

如果只谈功能不谈非功能，这个方案基本不算“可靠”。

14.1 性能要求

建议一期目标：

- 普通搜索响应：1~2 秒内
- Agent 检索接口：2~4 秒内返回候选结果
- 文档导入任务支持异步处理
- 批量任务支持队列机制

14.2 可用性要求

- 核心检索服务可用性目标：99.9% 起步
- 导入、同步、索引构建可异步重试

- 关键链路告警机制完善

14.3 安全要求

- 鉴权统一
- 接口签名 / Token 管理
- 重要操作审计留痕
- 数据分级控制
- 敏感字段脱敏预留

14.4 可观测性要求

- 查询日志
- 检索耗时
- 导入任务状态
- 审核流状态
- 失败重试记录
- Agent 引用记录

14.5 扩展性要求

- 数据源连接器可插拔
 - 检索策略可配置
 - 模板可扩展
 - 支持未来多租户演进
-

15. 推荐技术落地思路（不绑死具体厂商）

这里给你的是工程上可实现的技术路线，不是堆 buzzword。

15.1 整体技术分层

前端

- 管理后台 Web
- 运营工作台 Web
- 检索调试台 Web
- 业务使用端 Web / 内嵌业务系统页面

后端服务

建议拆成以下服务域：

- 用户与权限服务
- 知识内容服务
- 审核与流程服务
- 检索服务
- Agent 接入服务
- 数据源同步服务
- 任务调度服务
- 日志与分析服务

数据层

- 关系型数据库：存结构化业务数据
- 对象存储：存原始文件与附件

- 搜索引擎：存全文索引
- 向量库：存向量索引
- 缓存：提升查询性能
- 消息队列：处理导入、切片、索引、同步等异步任务

15.2 一期推荐实现策略

为了可实现，建议：

- 主库：关系型数据库
- 搜索：全文索引引擎
- 语义召回：向量检索
- 异步任务：消息队列 / 任务系统
- 文件处理：对象存储 + 文档解析服务

15.3 为什么不建议一期做成大一统超级服务

因为知识平台通常会出现：

- 导入处理慢
- 索引重建耗时长
- Agent 调用并发不稳定
- 分析统计写入量大

如果所有能力都塞进一个服务，后面性能和维护会比较惨。

16. 一期 MVP 方案（真正建议先做的）

这是最关键的一章。落地方案一定要有取舍。

16.1 一期建设目标

在 8~12 周内完成“知识录入—审核发布—检索服务—Agent 接入—引用展示—维护反馈”的最小闭环。

16.2 一期必须交付的能力

基础平台

- 登录与权限
- 知识空间管理
- 分类/标签管理
- 模板管理（先支持固定模板）

知识管理

- 手工录入
- FAQ / 产品说明 / SOP 三类模板
- 文件导入（PDF / Word / Markdown）
- 草稿 / 审核 / 发布
- 版本管理

检索服务

- 全文检索

- 语义检索
- 混合召回
- 基础重排
- 检索结果高亮

Agent 能力

- Agent 绑定知识空间
- 基础检索 API
- 引用字段返回
- 调试台

维护运营

- 过期提醒
- 反馈收集
- 维护任务
- 核心看板

安全与管理

- 角色权限
- 审计日志
- 数据源先支持本地/手工导入

16.3 一期不做或弱化

- 自动知识图谱
- 外部十几个连接器
- 多级复杂审核流

- 字段级精细权限
 - 高级 A/B 检索实验平台
 - 多租户 SaaS 完整版
-

17. 二期增强建议

当一期跑顺后，再做这些会更稳：

- Confluence / Notion / 飞书等连接器
 - 多模板与模板自定义
 - 复杂规则知识建模
 - 自动标签与自动摘要
 - 重复知识识别
 - 冲突知识检测
 - 检索评测集与效果评估系统
 - 多 Agent 策略管理
 - 灰度发布
 - API 对接第三方业务系统
-

18. 三期平台化建议

如果后续要把它做成集团级平台，可以考虑：

- 多租户能力
- 多模态知识

- 企业级合规中心
 - 跨区域部署
 - 插件市场 / 连接器市场
 - 开放平台与生态能力
-

19. 实施组织建议

方案可靠不可靠，很大程度取决于组织配合，不只是系统设计。

19.1 建议项目角色配置

- 产品负责人：1
- UI/交互：1
- 前端：1~2
- 后端：2~3
- 算法 / AI / 检索工程：1~2
- 测试：1
- 知识运营/业务代表：2~3（兼职也行，但必须有）

19.2 为什么一定要拉业务方进来

因为知识平台失败最常见原因不是技术不行，而是：

- 没人持续提供高质量知识
- 没人审核
- 没人维护

- 业务负责人不认这个平台

所以从项目启动就要明确每个知识域的 owner。

20. 实施节奏建议（12 周参考版）

第 1~2 周：需求与建模

- 明确场景和角色
- 定义知识对象与模板
- 定义信息架构与权限模型
- 确定 MVP 范围

第 3~4 周：原型与技术方案

- 完成核心页面原型
- 完成系统架构设计
- 明确数据模型与接口边界
- 制定开发计划

第 5~8 周：核心功能开发

- 后台管理
- 知识录入与审核
- 检索能力接入

- Agent 绑定与调试台
- 基础看板

第 9~10 周：联调与试运行

- 导入测试数据
- 调优检索效果
- 联调 Agent 引用链路
- 进行权限与审计验证

第 11~12 周：灰度上线

- 选 1~2 个业务域试点
- 跑维护闭环
- 收集效果数据
- 制定二期迭代项

21. 风险点与规避方案

风险一：平台做成了“高级文档库”

表现

- 只会上传文档
- 没有结构字段

- Agent 调用不精准

规避

- 必做模板化录入
- 必做结构化字段
- 必做 chunk 级召回

风险二：知识没人维护

表现

- 平台初期热闹，三个月后内容过期

规避

- 责任人机制
- 过期提醒
- 维护任务池
- 看板与考核

风险三：权限混乱导致泄露风险

规避

- 一期先做空间级 + 文档级权限
- Agent 侧加入权限过滤
- 全链路留审计日志

风险四：Agent 效果差，无法定位原因

规避

- 调试台必须上线
- Query log / citation log 必须有
- 检索结果和过滤原因必须可视化

风险五：项目过重，迟迟无法上线

规避

- 严格按 MVP 闭环推进
- 少做连接器，多做核心链路
- 先试点后扩展

22. 验收标准建议

要让这个平台可验收，不能只验“页面做没做完”，要验“闭环跑没跑通”。

22.1 功能验收

- 能创建知识空间
- 能按模板录入知识
- 能提交审核并发布
- 能被搜索到

- 能被 Agent 调用并返回引用
- 能记录反馈并形成维护任务

22.2 效果验收

建议一期试点目标：

- 重点问题检索命中率达到可用水平
- 关键知识更新时效显著改善
- 试点 Agent 回答引用率达到预期
- 业务试点用户反馈明显优于旧方式

22.3 管理验收

- 每个知识空间有 owner
- 有审核链路
- 有过期机制
- 有操作审计
- 有基础运营看板

23. 这份方案与你后续交付物的衔接关系

基于这份整体方案，后面可以顺着往下做，基本不会跑偏。

下一步可直接产出的内容

1. 功能架构图

把本方案中的 10 大模块画成结构图

2. 用户角色图

梳理管理员、运营、编辑、审核、Agent 配置人员、普通用户之间关系

3. 页面地图与原型蓝图

包括：

- 后台导航结构
- 核心页面清单
- 页面间跳转关系

4. PRD 文档

按模块详细展开需求、状态机、校验规则、异常流程

5. 字段设计

对 Space、Document、Chunk、AgentBinding、RetrievalPolicy 等对象逐一拆字段

6. Agent 联动展示页

重点做：

- Agent 绑定知识空间页
- 检索调试页
- Workflow 节点配置页
- 回答引用展示页

24. 最终建议：一条最稳的推进路径

如果要把这件事做成，而不是做热闹，我建议按这个顺序推进：

第一阶段：平台方案定版

产出：

- 整体方案
- 能力边界
- MVP 范围
- 角色与流程

第二阶段：产品设计

产出：

- 信息架构
- 页面地图
- 核心原型
- Agent 联动页面

第三阶段：PRD 与字段设计

产出：

- 模块级 PRD

- 对象字段表
- 状态流转
- 权限矩阵
- 埋点方案

第四阶段：技术落地设计

产出：

- 系统架构图
- 服务拆分
- 数据模型
- 接口设计
- 发布计划

第五阶段：试点上线

产出：

- 试点知识域上线
- Agent 联调
- 指标验收
- 二期规划

25. 当前版本结论

这套方案的核心不是“把文档集中”，而是：

把企业知识变成 Agent 可调用、业务可依赖、组织可治理、效果可度量的基础设施。

只要你按这个框架往下推进，后续无论是做：

- 交互原型图
- 产品设计图
- PRD 文档
- 字段设计
- Agent 链接展示

都会非常顺，不会东一榔头西一棒子。

26. 下一步我建议直接做什么

我建议下一步不要再泛泛讨论，直接进入可视化和产品化阶段。

最推荐的顺序是：

1. 先出《**功能架构图 + 用户角色图 + 页面地图**》
2. 再出《**核心页面原型蓝图**》
3. 再出《**PRD 总文档**》
4. 再出《**字段设计与 Agent 联动方案**》

如果你点头，下一轮我就直接开始给你：

《知识库平台功能架构图 + 用户角色图 + 页面地图（后台/运营台/调试台/业务端/Agent 集成端）》

这一步做完，整个产品骨架就真正立起来了。